CSE 141L Milestone 1

Jonathan Osorio, A17953638

Jacob Lam, A18519912

Academic Integrity

Your work will not be graded unless the signatures of all members of the group are present beneath the honor code.

To uphold academic integrity, students shall:

- Complete and submit academic work that is their own and that is an honest and fair representation of their knowledge and abilities at the time of submission.
- Know and follow the standards of CSE 141L and UCSD.

Please sign (type) your name(s) below the following statement:

I pledge to be fair to my classmates and instructors by completing all of my academic work with integrity. This means that I will respect the standards set by the instructor and institution, be responsible for the consequences of my choices, honestly represent my knowledge and abilities, and be a community member that others can trust to do the right thing even when no one is watching. I will always put learning before grades, and integrity before performance. I pledge to excel with integrity.

Jonathan Osorio

Jacob Lam

0. Team

   Jonathan Osorio

   Jacob Lam

1. Introduction

Our architecture is called ARIA (Accumulator-Register ISA Architecture). The overall design philosophy is to maximize instruction encoding efficiency within a strict 9-bit instruction width while retaining enough register flexibility to handle the nested-loop array processing required by all three target programs.

ARIA is best classified as a hybrid accumulator/load-store machine. Like a classic accumulator machine, all ALU operations implicitly write their result to a single fixed register (R0), requiring only one explicit register field per instruction and freeing up bits for a wider opcode space. Like a load-store machine, memory is only accessed through explicit load and store instructions — all computation happens exclusively in registers, and memory addresses are always register-indirect with a small offset.

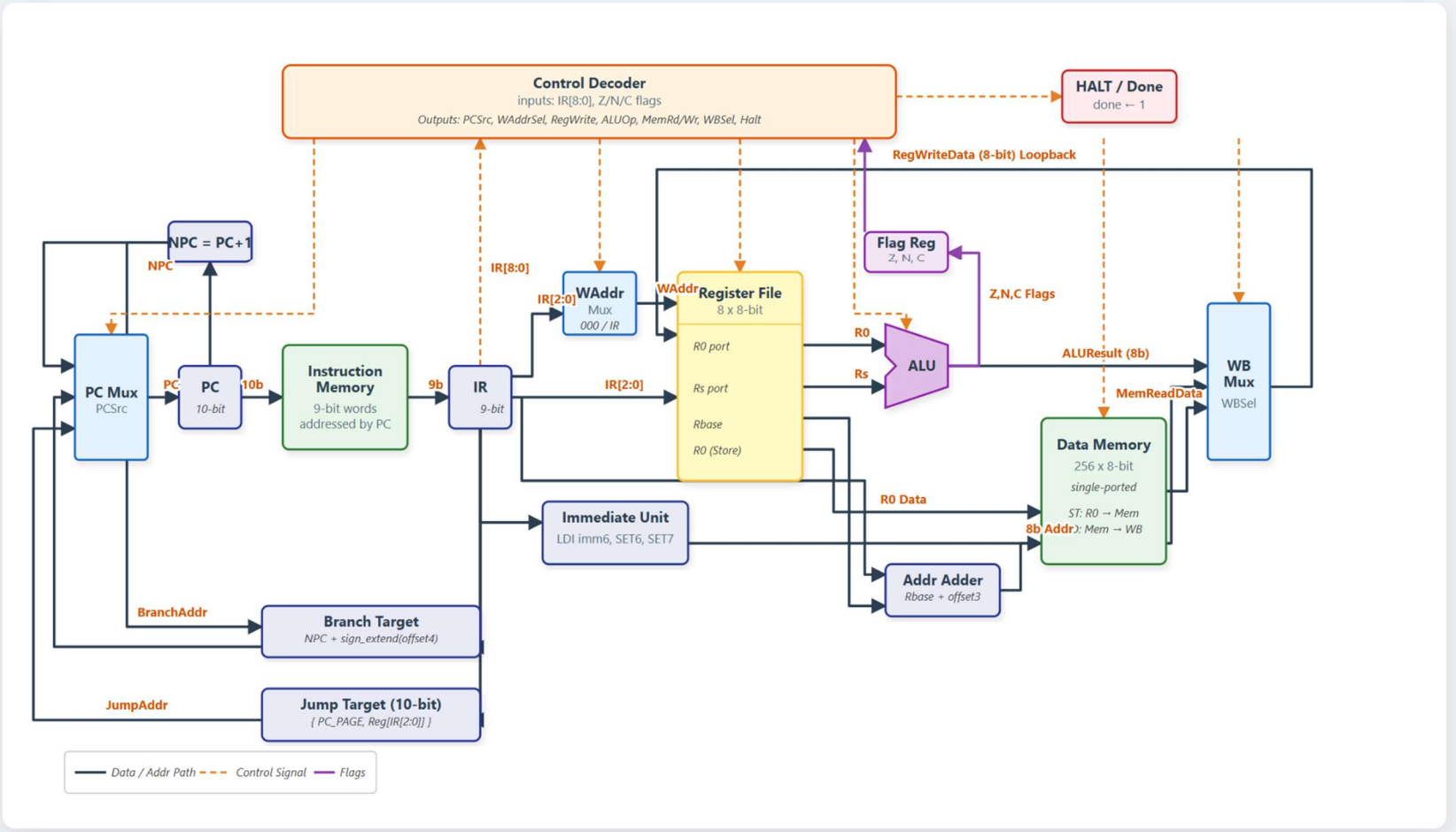
Our primary goals were: first, to fit a complete and practical instruction set within 9 bits without sacrificing the operations needed for XOR-based Hamming distance, multi-byte subtraction for arithmetic distance, and shift-and-add multiplication; second, to support 8 general-purpose registers so that loop counters, running min/max values, and operands can all remain in registers across the inner loop body without spilling to memory; and third, to keep the hardware implementation simple and regular, with a clean 2-bit format selector driving four distinct instruction formats and one implicit destination rule governing all ALU operations.

2. Architectural Overview

The diagram below shows the major hardware blocks of the ARIA processor and how they connect. The processor uses a Harvard architecture with separate instruction and data memories, satisfying the project requirement for independent instruction and data memory spaces.

Accumulator/Load-Store CPU Architecture

RTL-Derived Datapath • 9-bit Instruction • 8-bit Datapath • Single-Ported Memory



The major blocks are:

- Instruction Memory: A 9-bit wide ROM holding up to 1,024 assembled instructions. Loaded via $readmemb. Separate from data memory (Harvard).
- PC / Fetch Unit: A 10-bit program counter that increments each cycle, jumps on branches, and resets on start. Reads one 9-bit instruction per cycle from instruction memory.
- Control Decoder: Decodes the 9-bit instruction into control signals for every other block. Reads the top 2 bits (format selector), then format-specific fields for ALU op, register addresses, memory direction, branch condition, and done.
- Register File: Eight 8-bit registers (R0–R7). R0 is the accumulator — all ALU results write here. R1–R7 are general storage. The project requires at most one register write per instruction; this is satisfied by the accumulator design.
- ALU: Combinational logic performing all 16 Format A operations on R0 and one source register. Outputs an 8-bit result and three status flags: Z (zero), N (negative), C (carry).
- Flag Register: Holds Z, N, C flags from the last ALU operation. Read by the branch unit in the control decoder to determine whether to take a branch.
- Data Memory: A 256-byte single-ported RAM. The testbench writes input values into addresses 0–63 before start falls. Programs write results into addresses 64+ before asserting done via HALT.
- PC_PAGE Register: A 2-bit register set by the SETP instruction. Combined with an 8-bit register value by JR to form a 10-bit jump target, enabling jumps across the full 1,024-entry instruction space.

3. Machine Specification

ALU Operations

All ALU operations use format: 00 oooo rrr

R0 = accumulator; Rs = register specified by rrr

Name	Type	Bit Breakdown	Example	Notes
add	ALU	00 0000 rrr	ADD R2 → 000000010	R0 ← R0 + R2; updates Z, N, C
addc	ALU	00 0001 rrr	ADDC R5 → 000001101	R0 ← R0 + R5 + C; multi-byte addition

Name	Type	Bit Breakdown	Example	Notes
sub	ALU	00 0010 rrr	SUB R3 → 000010011	R0 ← R0 - R3; updates carry/borrow
subb	ALU	00 0011 rrr	SUBB R4 → 000011100	R0 ← R0 - R4 - C; multi-byte subtraction
and	ALU	00 0100 rrr	AND R1 → 000100001	Useful for masks and bit tests
or	ALU	00 0101 rrr	OR R1 → 000101001	Useful for combining bits

Name	Type	Bit Breakdown	Example	Notes
xor	ALU	00 0110 rrr	XOR R5 → 000110101	Essential for Hamming distance
cmp	ALU	00 0111 rrr	CMP R6 → 000111110	Flags for R0 - R6; does not overwrite R0
lsl	ALU	00 1000 000	LSL → 001000000	R0 ← R0 << 1; old bit 7 goes to C
lsr	ALU	00 1001 000	LSR → 001001000	R0 ← R0 >> 1; old bit 0 goes to C

Name	Type	Bit Breakdown	Example	Notes
rol	ALU	00 1010 000	ROL → 001010000	Rotate left through carry; useful for multi-byte shifts
ror	ALU	00 1011 000	ROR → 001011000	Rotate right through carry; useful for multi-byte shifts
not	ALU	00 1100 000	NOT → 001100000	R0 ← ~R0
neg	ALU	00 1101 000	NEG → 001101000	R0 ← -R0

Name	Type	Bit Breakdown	Example	Notes
clr	ALU	00 1110 000	CLR → 001110000	R0 ← 0
get	ALU	00 1111 rrr	GET R3 → 001111011	R0 ← R3

Memory Operations

Memory format: 01 m bbb qqq

Effective address = Rbbb + qqq (qqq is unsigned 3-bit offset, 0–7)

Name	Type	Bit Breakdown	Example	Notes
ld	Memory	01 1 bbb qqq	LD R3, 1 → 011011001	R0 ← mem[R3 + 1]
st	Memory	01 0 bbb qqq	ST R6, 4 → 010110100	mem[R6 + 4] ← R0

Branch Operations

Branch format: 10 ccc dddd

Target: if condition true, PC ← PC + 1 + sign_extend(dddd); else PC ← PC + 1

4-bit signed offset supports −8 to +7 instructions from PC+1

Name	Type	Bit Breakdown	Example	Notes
bz	Branch	10 000 dddd	BZ +3 → 100000011	Branch if Z=1
bnz	Branch	10 001 dddd	BNZ -4 → 100011100	Branch if Z=0

Name	Type	Bit Breakdown	Example	Notes
bn	Branch	10 010 dddd	BN -2 → 100101110	Branch if N=1
bnn	Branch	10 011 dddd	BNN +2 → 100110010	Branch if N=0
bc	Branch	10 100 dddd	BC +1 → 101000001	Branch if C=1
bnc	Branch	10 101 dddd	BNC +1 → 101010001	Branch if C=0
bra	Branch	10 110 dddd	BRA -7 → 101101001	Unconditional short branch
bnv	Branch	10 111 dddd	BNV +0 → 101110000	Useful as a branch-format NOP

Immediate Operations

Immediate format: 110 iiiiii

Name	Type	Bit Breakdown	Example	Notes
ldi	Immediate	110 iiiiii	LDI 37 → 110100101	R0 ← zero_extend(37); loads 0–63; use SET6/SET7 for larger values

Special Operations

Special format: 111 sss rrr

Name	Type	Bit Breakdown	Example	Notes
put	Special	111 000 rrr	PUT R4 → 111000100	R4 ← R0
jr	Special	111 001 rrr	JR R5 → 111001101	PC ← {PC_PAGE, R5}
halt	Special	111 010 000	HALT → 111010000	Asserts done=1
nop	Special	111 011 000	NOP → 111011000	Does nothing
set6	Special	111 100 000	SET6 → 111100000	R0 ← R0 | 0x40
set7	Special	111 101 000	SET7 → 111101000	R0 ← R0 | 0x80
setp	Special	111 110 rrr	SETP 2 → 111110010	PC_PAGE ← rrr[1:0]
clc	Special	111 111 000	CLC → 111111000	C ← 0

Internal Operands

Registers

The processor supports 8 registers, R0–R7. Each register is 8 bits.

Register	Role
R0	Accumulator
R1	General register, often loop counter
R2	General register, often inner loop counter
R3	General register, often input pointer
R4	General register, temporary byte/value
R5	General register, temporary byte/value
R6	General register, often min/max or output pointer
R7	General register, often output pointer/scratch

The processor supports 8 registers, each 8 bits wide, labeled R0 through R7. R0 is special: it is the accumulator and the implicit destination for all Format A (ALU) instructions. R1–R7 are general-purpose storage registers with no hardware-imposed special behavior.

Flags

The processor also maintains three status flags updated by ALU operations:

Flag	Meaning
Z	Result was zero
N	Result was negative (bit 7 of result is 1)
C	Carry for addition, borrow for subtraction, or shifted-out bit for shifts and rotates

Branches use these flags. The processor also maintains three status flags updated by ALU operations: Z (result equals zero), N (result bit 7 is set, indicating a negative two's-complement value), and C (carry out from addition, borrow from subtraction, or shifted-out bit from shifts and rotates).

Control Flow (Branches)

Eight branch conditions are supported using the Format C instruction (10 | ccc | dddd):

cond [6:4]	Mnemonic	Condition	Use Case
000	BZ	Z = 1 (result was zero)	Loop termination, equality check
001	BNZ	Z = 0 (result was not zero)	Inner loop control
010	BN	N = 1 (result was negative)	Detect negative difference (absolute value)
011	BNN	N = 0 (result was non-negative)	General comparison
100	BC	C = 1 (carry/borrow set)	Multi-byte arithmetic, popcount shifts
101	BNC	C = 0 (no carry)	Multi-byte arithmetic

cond [6:4]	Mnemonic	Condition	Use Case
110	BRA	Always	Unconditional short jump
111	BNV	Never	No-op in branch format

Branch target: PC ← NPC + sign_extend(offset4)

Large jump: PC ← {PC_PAGE, Reg[IR[2:0]]}

Branching and Jumping

Control flow is divided into two methods based on distance: Short Conditional Branches for tight loops, and Long Register Jumps for moving anywhere else in the 1024-instruction memory space.

Standard conditional branches (like BZ and BNZ) use a 4-bit signed offset:

• Calculation: target = PC + 1 + sign_extend(offset4)

• Range: −8 to +7 instructions from the instruction immediately following the branch.

Unconditional Long Jumps: To jump anywhere in the 1024-instruction memory limit, construct a full 10-bit address using an absolute jump (JR). Because instruction widths are small, you cannot simply write JUMP 300. You must construct the address piece by piece: load the lower bits into a register, set the upper “page” bits, and jump.

Scenario	Method
Short distance, conditional	Use standard BZ/BNZ directly
Long distance, unconditional	Build 10-bit address using LDI, PUT, SETP, and JR
Long distance, conditional	Build the address, use a short branch to conditionally skip over it, and execute the JR

Addressing Modes

The only memory addressing mode is base-register plus 3-bit unsigned offset:

effective_address = R_base + offset   (8-bit result, 0–255)

The base register (bbb field) holds a data memory address. The offset (qqq field, 0–7) is added unsigned. R0 is the implicit data register for both loads and stores.

Examples:

```
; Load high and low bytes of a 16-bit value at address in R3:
LD R3, 0   ; R0 <- mem[R3 + 0]  (high byte)
PUT R4     ; R4 = high byte
LD R3, 1   ; R0 <- mem[R3 + 1]  (low byte)
PUT R5     ; R5 = low byte

; Store a 32-bit product at address in R7:
GET R1   ; R0 = byte 3 (MSB)
ST R7, 0 ; mem[R7 + 0] = MSB
GET R2
```

```
ST R7, 1

GET R3

ST R7, 2

GET R4   ; R0 = byte 0 (LSB)

ST R7, 3 ; mem[R7 + 3] = LSB


; Building addresses above 63 (output regions at 64–127):

LDI 0    ; R0 = 0

SET6     ; R0 = 0 | 0x40 = 64

PUT R6   ; R6 = 64  (base of output region)
```

4. Programmer's Model [Lite]

The programmer should think of ARIA as a small accumulator machine with helper registers and clean load / store memory access. The mental model is: R0 is the workbench, R1-R7 are shelves, and data memory is long-term storage.

The general strategy is as follows. Use LDI, SET6, and SET7 to construct constants and addresses at program startup, then park them in R1-R7 using PUT. Load pairs of bytes from memory into R0, immediately moving them to shelf registers before loading the next byte. Perform all arithmetic and logic through R0, for 16-bit values, process the low byte first with ADD/SUB, then the high byte with ADDC/SUBB to propagate carry. For multi-byte shifts, use ROR/ROL through carry across successive 8-bit chunks. Use CMP followed by a branch for min/max decisions. For loops, preload the loop target address into a register at startup and use the inverse-branch + JR idiom for any loop body that may exceed 8 instructions. Store final answers into data memory before executing HALT, ensuring done is asserted only after all results are written.

We cannot directly copy MIPS or ARM instructions. Both architectures assume 32-bit instruction widths and can afford to name two or three register operands per instruction. With 8 registers requiring 3 bits each, three register fields would cost 9 bits — leaving zero bits for the opcode. Even two explicit register fields leave only 3 bits for opcodes, giving just 8 operations — insufficient for our needs.

We adapt the high-level ideas: load / store memory discipline, condition-flag-based branches, and ALU operations like XOR, shifts, and add-with-carry. The key departure is making the destination implicit (always R0), which is the single compromise that makes the 9-bit budget work without sacrificing opcode coverage.

4.3 "Will your ALU be used for non-arithmetic instructions (e.g., MIPS or ARM-like memory address pointer calculations, PC relative branch computations, etc.)? If so, how does that complicate your design?".

Yes, the ALU performs non-arithmetic operations, but no, it is not used for address calculations or branch computations.

What the ALU handles:

- Logical Operations: AND, OR, XOR, NOT - used for bit manipulation (e.g., XOR for Hamming distance in Program 1)
- Shift/Rotate Operations: LSL, LSR, ROL, ROR - critical for:
    - Multi-byte arithmetic (ROL/ROR through carry in Program 3's 32-bit shifts)
    - Address doubling (LSL for multiplying by 2)
    - Bit extraction in multiplication
- Data Movement: GET (copy register to R0), CLR (zero R0)

5. Individual Component Specification

Instruction memory is separate from data memory, R0 is the accumulator, R1–R7 are general-purpose registers, and the architectural status flags are Z, N, and C.

Top Level

Module file name: rtl/DUT.sv | Module name: DUT

Functionality Description

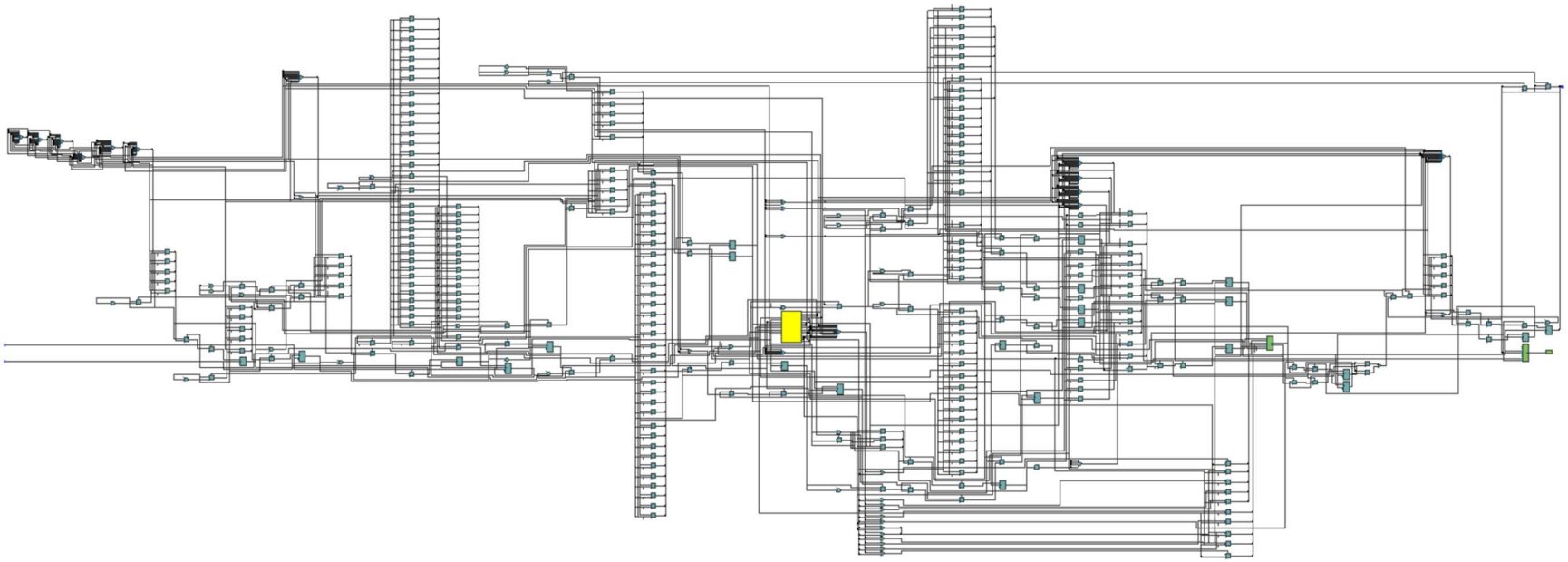
The top-level module connects the processor-visible blocks and exposes the testbench interface:

```
module DUT #(
  parameter int PROGRAM_ID = 1
)(
  input  logic clk,
  input  logic start,
  output logic done
);
```

start is the reset/restart input. When start is high, the processor architectural state is reset and done is cleared. Execution begins after start goes low. done is a registered completion signal that is asserted only after the selected program has finished writing its required output bytes.

The top level instantiates the data memory as dm, so the exported testbenches can preload input data and inspect output data through D1.dm.core. It also instantiates the program counter and instruction memory blocks used by the ARIA fetch path. Program selection is controlled by the PROGRAM_ID parameter or by compile-time defines PROG1, PROG2, and PROG3.

Schematic: *Insert top-level schematic here. Should show: clk, start, done; program counter/fetch path; instruction memory; control/decode path; register file; ALU and flag path; data memory instance dm; writeback and memory-address muxing.*



Program Counter

Module file name: rtl/aria_pc.sv | Module name: aria_pc | Testbench: tb/pc_tb.sv

Functionality Description

The program counter module stores the current 10-bit instruction address (supports up to 1024 instruction memory entries). On start, the PC resets to zero. During normal execution, the PC increments by one each cycle. Two control flows are supported:

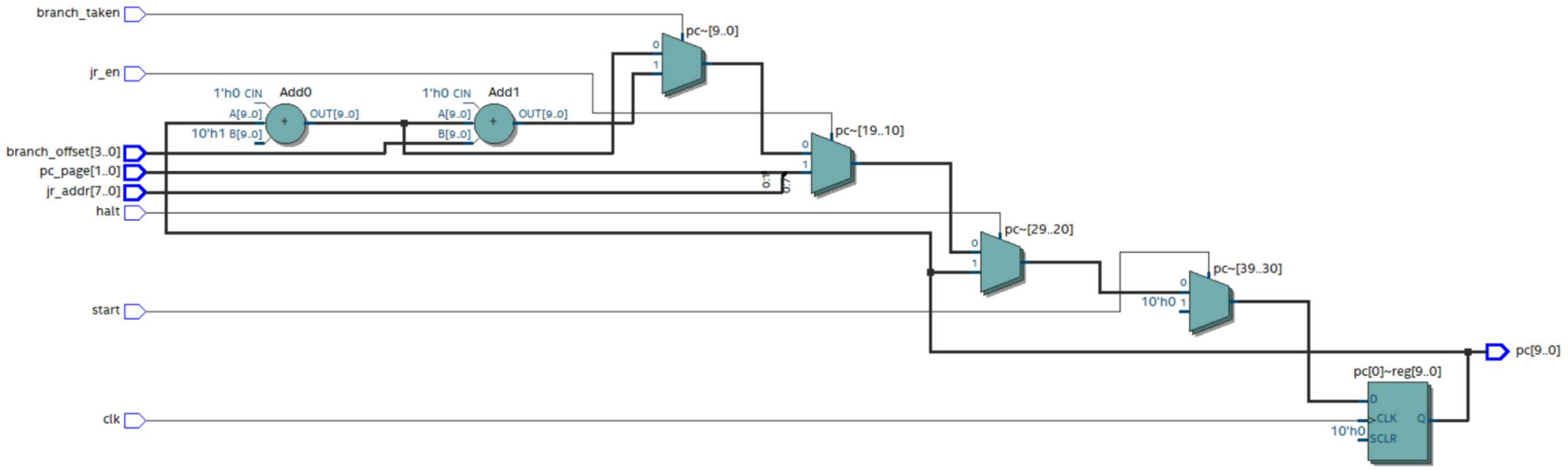
Mode	Operation
Short signed branch	PC ← PC + 1 + sign_extend(offset4); range −8 to +7 from PC+1
Register jump	PC ← {PC_PAGE, jr_addr}; combines 2-bit PC_PAGE with 8-bit register value

When halt is asserted, the PC holds its current value.

Testbench Description

tb/pc_tb.sv verifies: reset to address zero while start is asserted, sequential PC increment, positive signed branch offset, negative signed branch offset, branch-not-taken behavior, register jump using PC_PAGE, halt hold behavior, 10-bit wraparound from 10'h3ff to 10'h000.

Schematic: *Insert program counter schematic here. Should include: PC register, incrementer, signed branch adder, JR target concatenation, next-PC mux, and reset/halt controls.*



Timing Diagram: *Insert timing diagram screenshot here. Should demonstrate: reset, increment, branch, JR, halt hold, and wraparound.*

Instruction Memory

Module file name: rtl/aria_imem.sv | Module name: aria_imem

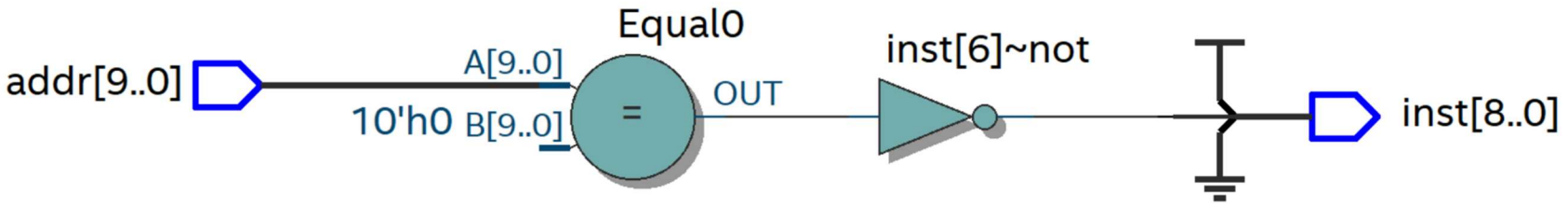
Functionality Description

Instruction memory is a ROM addressed by the 10-bit program counter. Each instruction is 9 bits wide, with a limit of 1024 entries.

Unused entries should decode as HALT or NOP.

Prefix	Format	Meaning
00	Format A	ALU operation
01	Format B	Memory load/store
10	Format C	Branch
110	Format D	6-bit immediate load
111	Format E	Special operation

Schematic: *Insert instruction memory schematic here. Should show: 10-bit address input, 9-bit instruction output, and the selected program image.*



Control Decoder

Module file name: rtl/aria_decoder.sv | Module name: aria_decoder

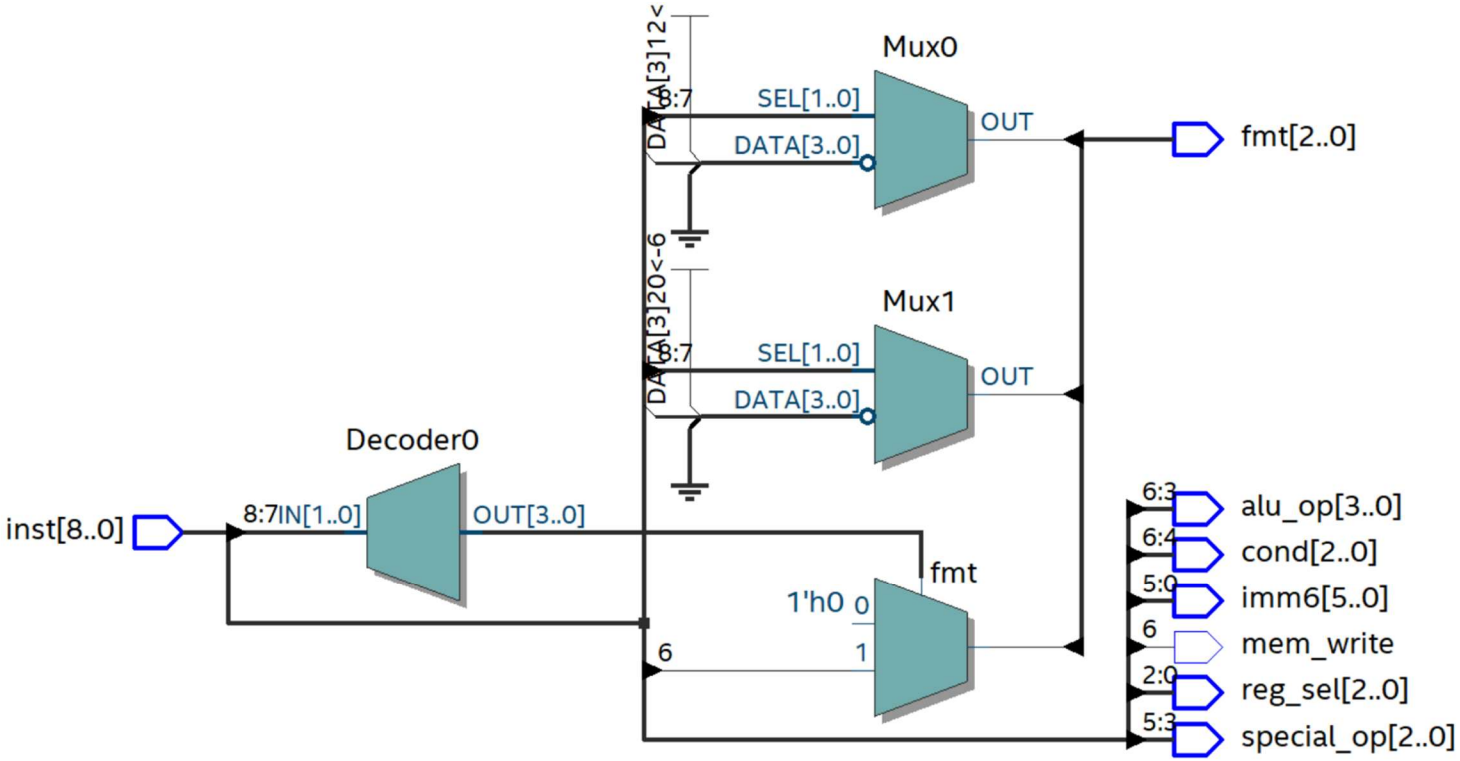
Functionality Description

The control decoder interprets the 9-bit instruction and generates control fields. Decode is hierarchical: first inspect inst[8:7], then differentiate immediate and special instructions when the prefix is 11.

The decoder exposes: instruction format, ALU opcode oooo, register selector rrr, branch condition ccc, immediate field iiiiii, special opcode sss, memory direction bit m.

Format	Encoding	Fields
ALU	00 oooo rrr	ALU operation and source register
Memory	01 m bbb qqq	Load/store, base register, unsigned offset
Branch	10 ccc dddd	Branch condition and signed offset
Immediate	110 iiiiii	Zero-extended 6-bit immediate
Special	111 sss rrr	Special operation and register field

Schematic: *Insert control decoder schematic here. Should show: format decode, opcode decode, control-signal generation, and branch-condition decode.*



Register File

Module file name: rtl/aria_reg_file.sv | Module name: aria_reg_file

Functionality Description

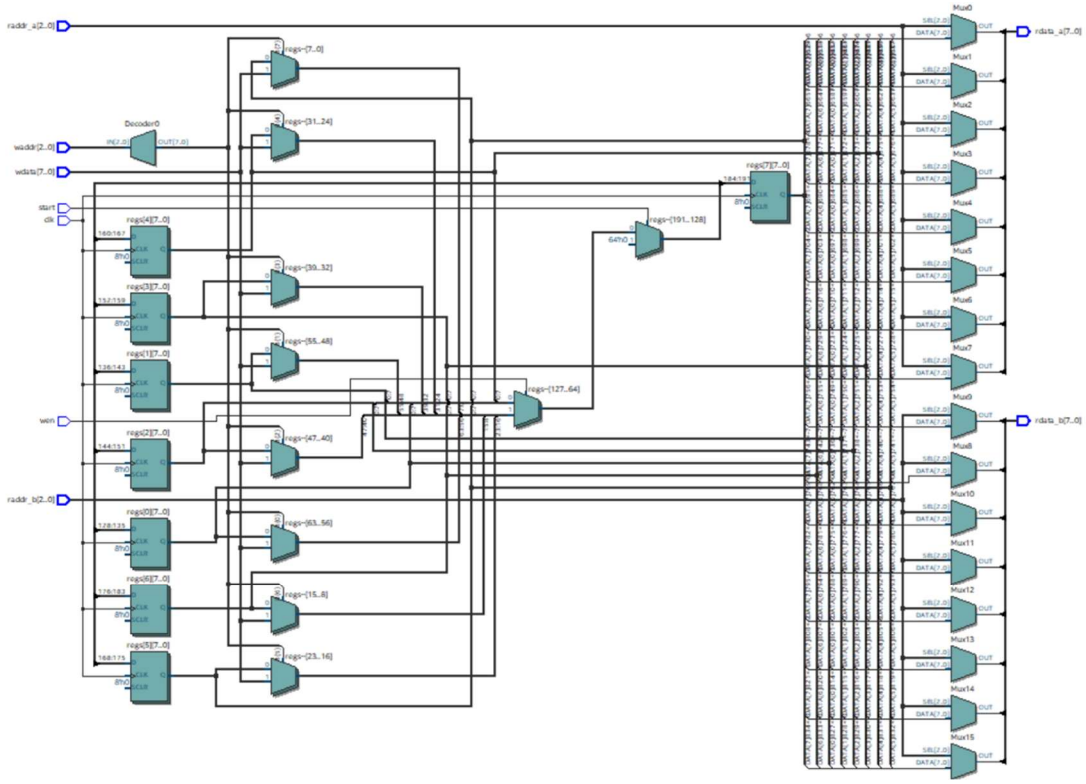
The register file contains eight 8-bit registers, R0 through R7. R0 is the accumulator and is the implicit destination for Format A ALU instructions. R1 through R7 are general-purpose registers used for loop counters, pointers, temporary byte storage, min/max values, output pointers, and scratch values.

Supports: 2 asynchronous read ports, 1 synchronous write port, reset of all registers when start is asserted, and one register write per cycle. This accounts for the constraint of no more than two register reads and one register write per instruction.

Register	Role
R0	Accumulator
R1	General register, often loop counter
R2	General register, often inner loop counter
R3	General register, often input pointer
R4	General register, temporary byte/value
R5	General register, temporary byte/value

Register	Role
R6	General register, often min/max or output pointer
R7	General register, often output pointer/scratch

Schematic: *Insert register file schematic here. Should show: the eight registers, two read address ports, one write address port, write enable, write data, and read data outputs.*



ALU (Arithmetic Logic Unit)

Module file name: rtl/aria_alu.sv | Module name: aria_alu | Testbench: tb/alu_tb.sv

Functionality Description

The ALU is combinational logic operating on the accumulator value R0, one source register value Rs, and the incoming carry flag. It produces an 8-bit result and three flags. Most ALU operations write the result back to R0. CMP is the exception: it computes flags for R0 - Rs but does not overwrite R0.

Flag	Meaning
Z	Result is zero
N	Result bit 7 is set
C	Carry for addition, borrow for subtraction, shifted-out bit for shifts/rotates

ALU Operations

Opcode	Mnemonic	Operation	Notes
0000	ADD	R0 + Rs	Updates Z, N, C
0001	ADDC	R0 + Rs + C	Multi-byte addition
0010	SUB	R0 - Rs	C means borrow
0011	SUBB	R0 - Rs - C	Multi-byte subtraction
0100	AND	R0 & Rs	Bit masks/tests
0101	OR	R0 | Rs	Bit combining

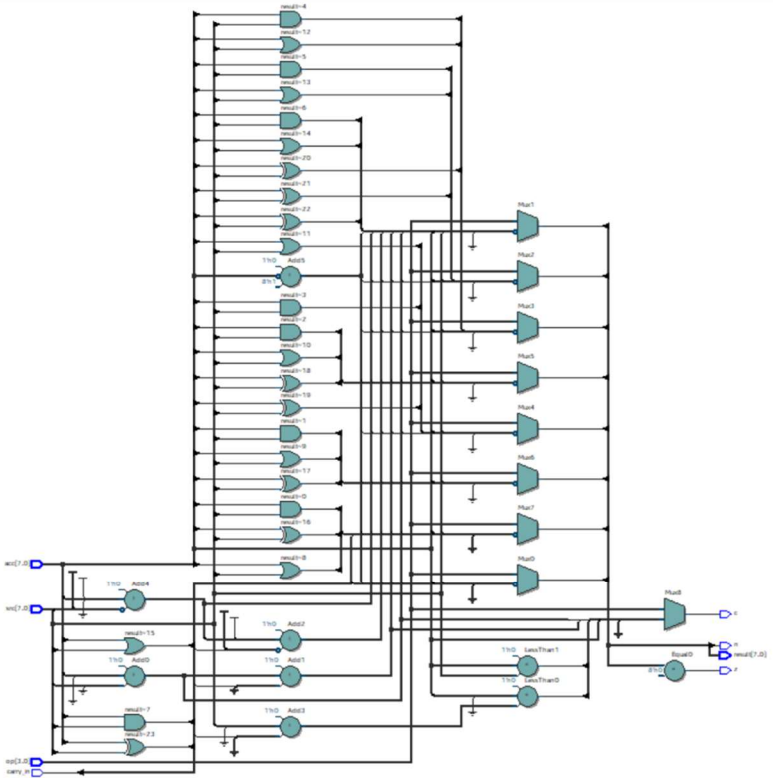
Opcode	Mnemonic	Operation	Notes
0110	XOR	R0 ^ Rs	Hamming distance support
0111	CMP	flags for R0 - Rs	No R0 writeback
1000	LSL	R0 << 1	Old bit 7 goes to C
1001	LSR	R0 >> 1	Old bit 0 goes to C
1010	ROL	Rotate left through C	Old C enters bit 0
1011	ROR	Rotate right through C	Old C enters bit 7

Opcode	Mnemonic	Operation	Notes
1100	NOT	~R0	Bitwise invert
1101	NEG	-R0	Two’s-complement negate
1110	CLR	0	Clear accumulator
1111	GET	Rs	Copy register into accumulator

Testbench Description

tb/alu_tb.sv applies representative operands to every ALU opcode and checks the result and flags. Tests include: carry in, borrow, zero result, negative result, shifts, rotates through carry, compare behavior, and register pass-through.

Schematic: *Insert ALU schematic here. Should show: arithmetic, logic, shift/rotate, and pass-through result paths feeding a result mux, plus flag-generation logic.*



Timing Diagram: *Insert ALU timing diagram screenshot here. Should demonstrate all 16 ALU operations and the resulting Z/N/C flags.*

Data Memory

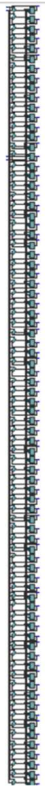
Module file name: rtl/dat_mem.sv | Module name: dat_mem

Functionality Description

Data memory is a 256-byte RAM with one address port, with asynchronous read and synchronous write behavior. The array is declared as logic [7:0] core[256] inside the dat_mem module (required for testbench path D1.dm.core).

Address Range	Contents
core[0:63]	32 input 16-bit operands loaded by testbench
core[64–65]	Program 1 output: min/max Hamming distance
core[66:69]	Program 2 output: min/max arithmetic distance
core[64:127]	Program 3 output: sixteen 32-bit products
core[128:255]	Available as scratch space

Schematic: *Insert data memory schematic here. Should show: 8-bit address port, 8-bit data input, 8-bit data output, write enable, clock, and 256-byte storage array.*



Lookup Tables

Module file name: N/A

Functionality Description

No dedicated lookup tables are needed for ARIA. The control path uses direct field decode from the 9-bit instruction, and the ALU implements operations directly with combinational arithmetic, logic, and shift/rotate hardware. Small decode tables may be

represented as case statements for ALU opcode selection, branch condition selection, and special instruction selection (well below the limit of 32).

Schematic: *No lookup-table schematic is required unless the final implementation introduces a separate lookup-table module.*

Muxes

Module file name: Integrated into datapath modules

Functionality Description

The processor uses muxes in several datapath locations:

Mux	Selects Between
Next-PC mux	Sequential PC, branch target, JR target, or hold

Mux	Selects Between
ALU result mux	Arithmetic, logic, shift/rotate, clear, negate, or register pass-through
Writeback mux	ALU result, load data, immediate data, or special-operation result for R0
Data-memory address mux	Base-register-plus-offset address for loads and stores

Schematic: *Insert mux/datapath schematic here if shown separately from the top-level schematic.*

Branch Condition Logic

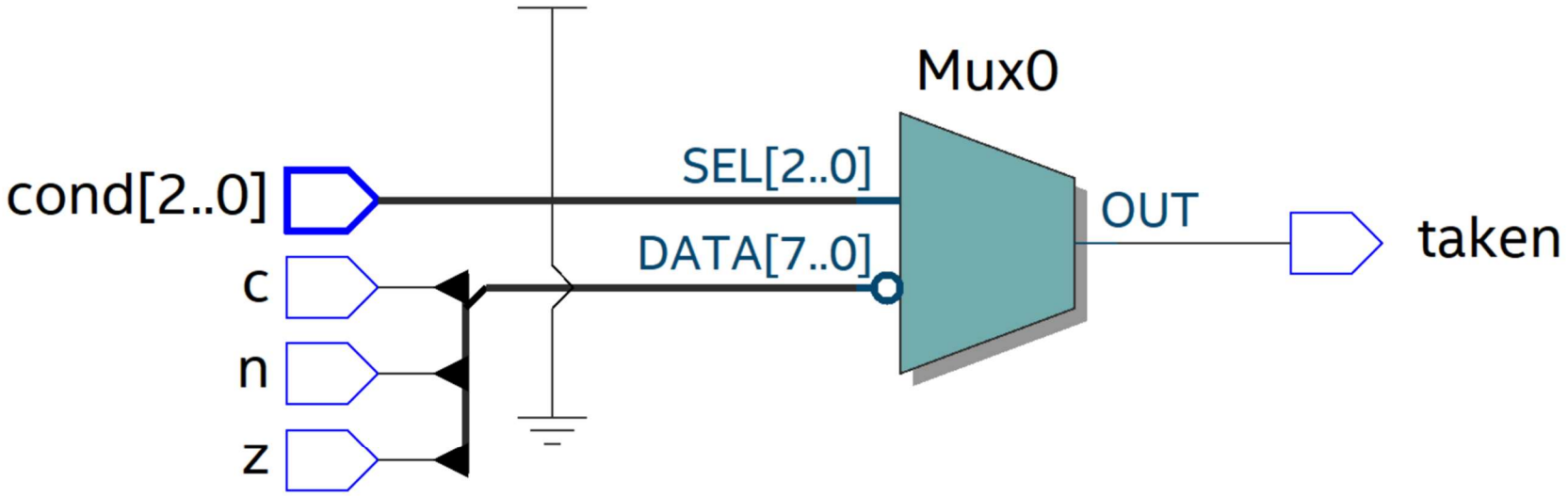
Module file name: rtl/aria_branch.sv | Module name: aria_branch

Functionality Description

The branch condition logic evaluates the 3-bit branch condition field against the architectural flags Z, N, and C. The branch unit feeds the program-counter branch-taken input.

Condition Field	Mnemonic	Taken When
000	BZ	Z = 1
001	BNZ	Z = 0
010	BN	N = 1
011	BNN	N = 0
100	BC	C = 1
101	BNC	C = 0
110	BRA	Always
111	BNV	Never

Schematic: *Insert branch-condition logic schematic here. Should show: Z/N/C inputs, condition decode, and the branch-taken output.*



PC_PAGE Register

Module file name: Integrated into rtl/DUT.sv

Functionality Description

PC_PAGE is a 2-bit register written by the SETP special instruction. Register jumps form a 10-bit target by concatenating PC_PAGE with an 8-bit register value: PC ← {PC_PAGE, R[rrr]}. This allows the processor to reach all 1024 instruction memory entries even though each instruction is only 9 bits.

Schematic: *Insert PC_PAGE schematic here or include it as part of the PC/fetch schematic.*

6. 6. Program Implementation

Program 1 Pseudocode

```
min_dist = 16
max_dist = 0

for j = 0 to 31:
   for k = j + 1 to 31:
      A = load16(2 * j)
      B = load16(2 * k)
```

```
        xor_value = A XOR B
        dist = popcount16(xor_value)

        if dist < min_dist:
            min_dist = dist

        if dist > max_dist:
            max_dist = dist

mem[64] = min_dist
mem[65] = max_dist
halt

popcount16(x):
    count = 0
    loop_counter = 16
    while loop_counter != 0:
```

```
        if x[0] == 1:
            count += 1
        x = x >> 1
        loop_counter -= 1
    return count
```

Program 1 Assembly Code

```
; --- Program 1 startup: build constants and pointers ---
    LDI  16         ; R0 = 16
    PUT  R6         ; R6 = min_dist = 16 (max possible)
    CLR             ; R0 = 0
    PUT  R7         ; R7 = max_dist = 0
    CLR
    PUT  R1         ; R1 = i = 0

; outer loop: for i = 0 to 31
outer:
    GET  R1
```

```
    PUT  R2        ; R2 = j = i
    LDI  1
    ADD  R2
    PUT  R2        ; R2 = j = i+1

; inner loop: for j = i+1 to 31
inner:
; load A = mem[2*i], mem[2*i+1]
    GET  R1
    LSL
    PUT  R3        ; R3 = 2*i
    LD   R3, 0     ; R0 = A_high
    PUT  R4
    LD   R3, 1     ; R0 = A_low
    PUT  R5
; load B = mem[2*j], mem[2*j+1]
    GET  R2
```

```
    LSL
    PUT  R3        ; R3 = 2*j
    LD   R3, 0     ; R0 = B_high
    XOR  R4        ; R0 = A_high XOR B_high
    PUT  R4
    LD   R3, 1     ; R0 = B_low
    XOR  R5        ; R0 = A_low XOR B_low
    PUT  R5
; popcount on {R4, R5} -> accumulate in a counter register
    ; ... popcount loop (16 iterations, shift and count) ...
; compare to min/max
    ; ... update R6 (min) and R7 (max) using CMP + BNN/BN ...
; j++
    LDI  1
    ADD  R2
    PUT  R2
    LDI  32
```

```
    CMP  R2
    BNZ  inner   ; branch if j != 32
; i++
    LDI  1
    ADD  R1
    PUT  R1
    LDI  32
    CMP  R1
    BNZ  outer   ; branch if i != 32
; store results
    LDI  0
    SET6         ; R0 = 64
    PUT  R3      ; R3 = 64
    GET  R6
    ST   R3, 0   ; mem[64] = min_dist
    GET  R7
    ST   R3, 1   ; mem[65] = max_dist
```

```
    HALT
```

Program 2 Pseudocode

```
min_dist = 0xFFFF
max_dist = 0
for j = 0 to 31:
    for k = j + 1 to 31:
        A = signed_load16(2 * j)
        B = signed_load16(2 * k)
        diff = sign_extend_17(A) - sign_extend_17(B)
        if diff < 0:
            diff = -diff
        dist = lower16(diff)
        if dist < min_dist:
            min_dist = dist
        if dist > max_dist:
            max_dist = dist
store16(66, min_dist)
```

store16(68, max_dist)

halt

Program 2 Assembly Code

```
; 16-bit subtraction: {R4,R5} - {R6,R7} -> {R4,R5}
;   GET R5 / SUB R7 / PUT R5   (low byte, sets borrow in C)
;   GET R4 / SUBB R6 / PUT R4  (high byte, uses borrow)


; Absolute value: test sign bit of high byte (R4)
;   GET R4 / LSL           (shift sign into C)
;   BNC  positive          (if C=0, already positive)
;   GET R5 / NEG / PUT R5      (negate low byte)
        ;   GET R4 / NEG / ADDC R0 ...  (negate high byte + carry)


        ; Min/max compare is same structure as Program 1
```

Program 3 Pseudocode

```
        for k = 0 to 15:
           A = signed_load16(4 * k)
```

```
B = signed_load16(4 * k + 2)
sign = sign(A) XOR sign(B)
if A < 0:
   A = -A
if B < 0:
   B = -B
product = 0
multiplicand = zero_extend_32(A)
multiplier = B
counter = 0
while counter != 16:
   if multiplier[0] == 1:
      product = product + multiplicand
   multiplicand = multiplicand << 1
   multiplier = multiplier >> 1
   counter += 1
if sign == 1:
```

```
            product = -product
        store32(64 + 4 * k, product)
    halt
```

Program 3 Assembly Code

```
    ; 32-bit left shift of {P3,P2,P1,P0} using ROL through carry:
    ;   CLR / GET P0 / ROL / PUT P0   (shift LSB, old bit7->C)
    ;   GET P1 / ROL / PUT P1         (C->bit0, old bit7->C)
    ;   GET P2 / ROL / PUT P2
    ;   GET P3 / ROL / PUT P3         (MSB)

    ; 32-bit addition {P3,P2,P1,P0} += {M3,M2,M1,M0}:
    ;   CLC
    ;   GET P0 / ADDC M0 / PUT P0
    ;   GET P1 / ADDC M1 / PUT P1
    ;   GET P2 / ADDC M2 / PUT P2
    ;   GET P3 / ADDC M3 / PUT P3
```

; Outer loop: k = 0..15, increment output pointer by 4

; Inner loop: bit = 0..15, test LSB of multiplier each time

7. Changelog

- Milestone 3
- Milestone 2
- Milestone 1